Theory of Automata
CS-3005
Lecture 7

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

NFA - Non-Deterministic Finite Automata

- Definition: An NFA is a TG with a unique start state and with the property that each of its edge labels is a single alphabet letter.
- The regular deterministic finite automata are referred to as DFAs, to distinguish them from NFAs.
- As a TG, an NFA can have arbitrarily many a-edges and arbitrarily many b-edges coming out of each state.
- An input string is accepted by an NFA if there exists any possible path from - to +.

NFA

- An NFA is quintuple $M=\{Q,\Sigma,q_0, F, \delta\}$
  - Q is set of finite states
  - Σ is a finite set of symbols called ***alphabet***
  - q_0 belongs to Q is distinguished ***Start State***
  - F is subset of Q called the ***Final*** or Accepting states
  - δ is a total function from $Q \times \Sigma$ to P(Q) known as ***transition function***, such that, an input symbol may cause more than one next states, i.e., to one state out of a set of possible next states.
  - P(Q) is the power set of Q, that is, 2^Q

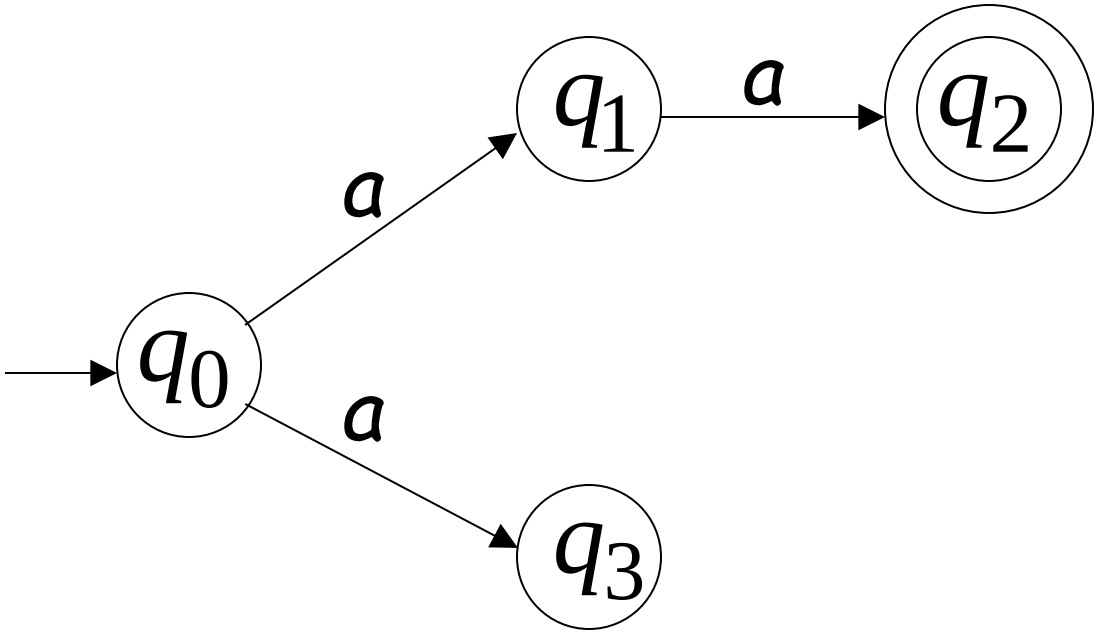
NFA

Definition: A nondeterministic finite automaton (or NFA) is a TG with a unique start state and with the property that each of its edge labels is a single alphabet letter.

- The regular deterministic finite automata are referred to as DFAs, to distinguish them from NFAs.
- As a TG, an NFA can have arbitrarily many a-edges and arbitrarily many b-edges coming out of each state.
- An input string is accepted by an NFA if there exists any possible path from - to +.

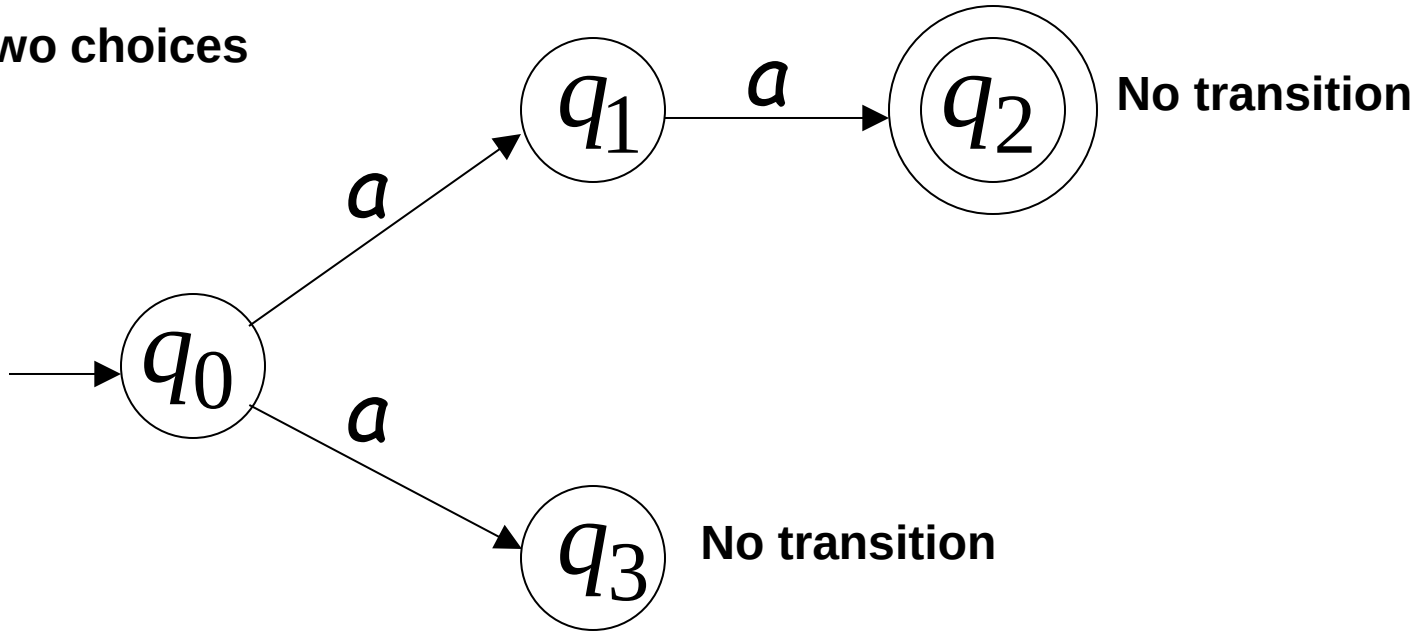
NFA

Alphabet = $\{a\}$

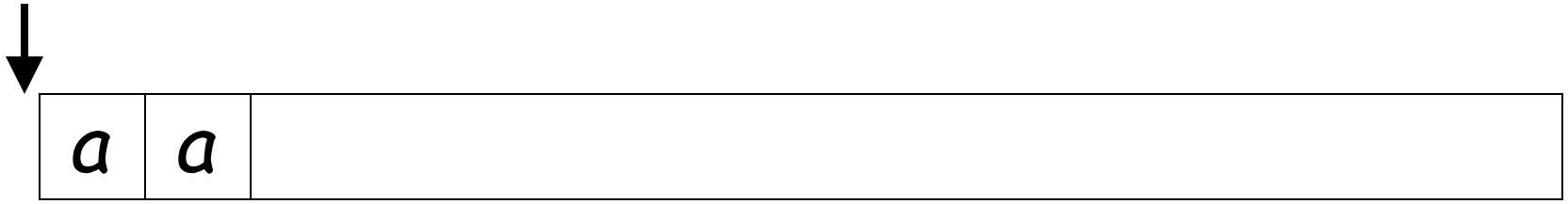


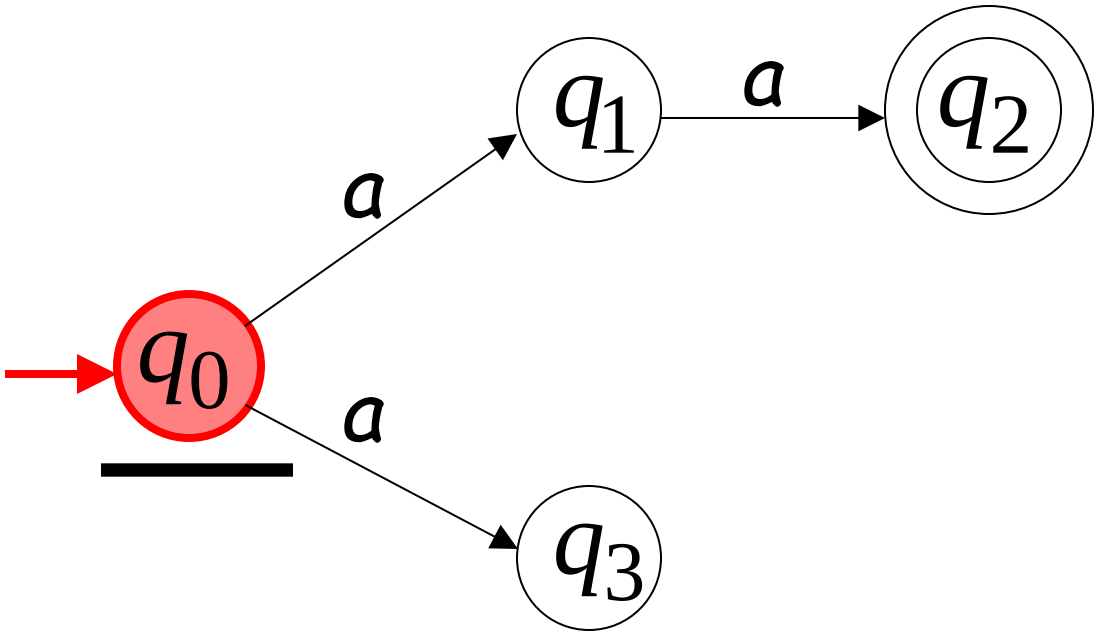
NFA

Alphabet = $\{a\}$

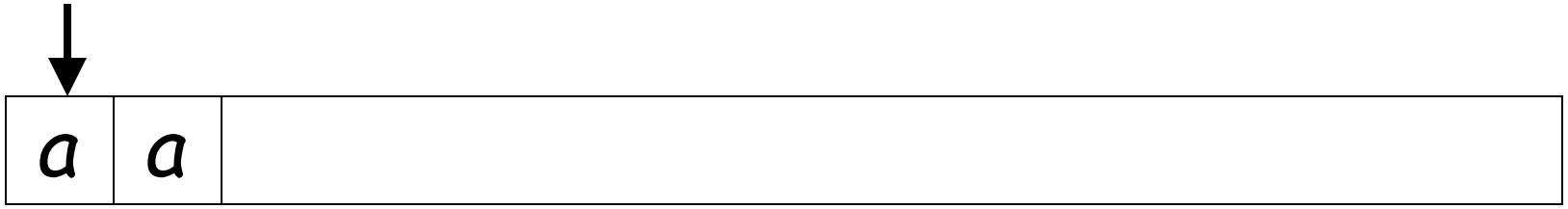


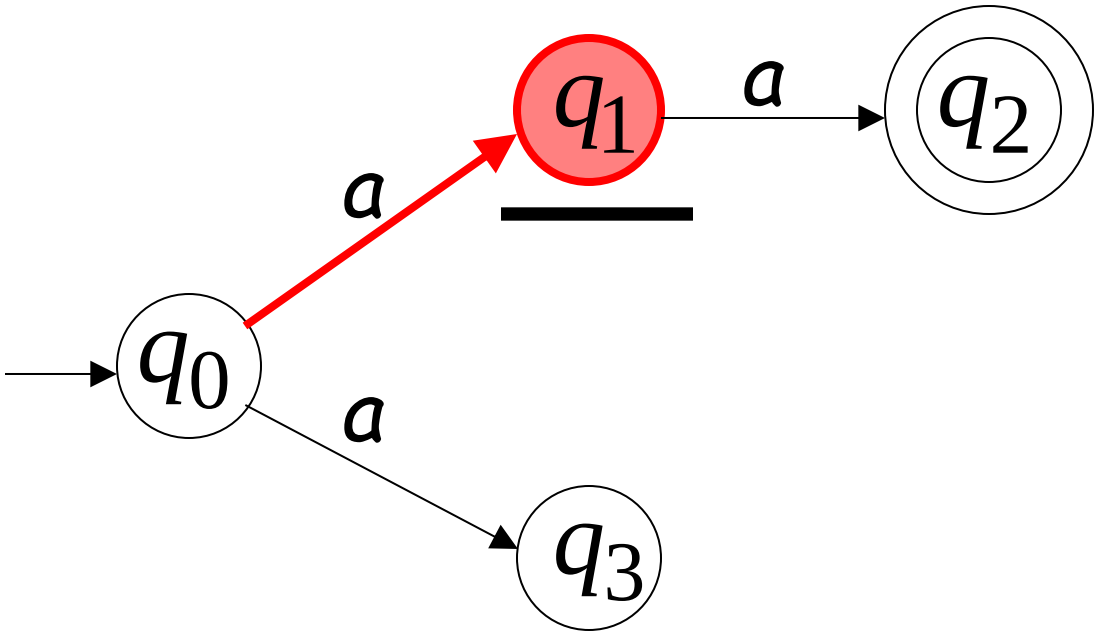
First Choice



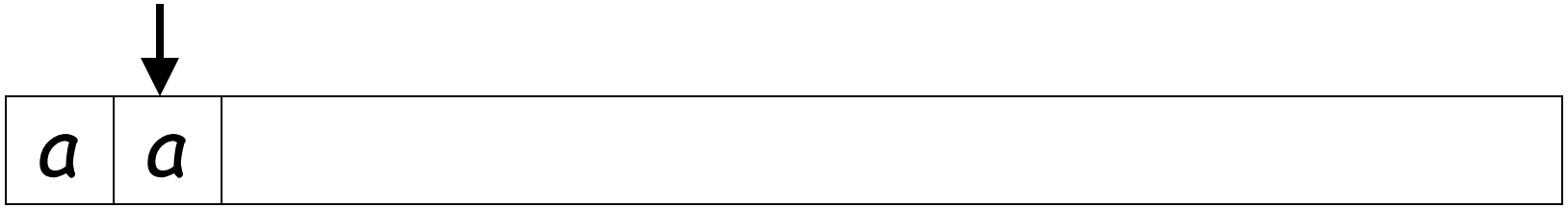


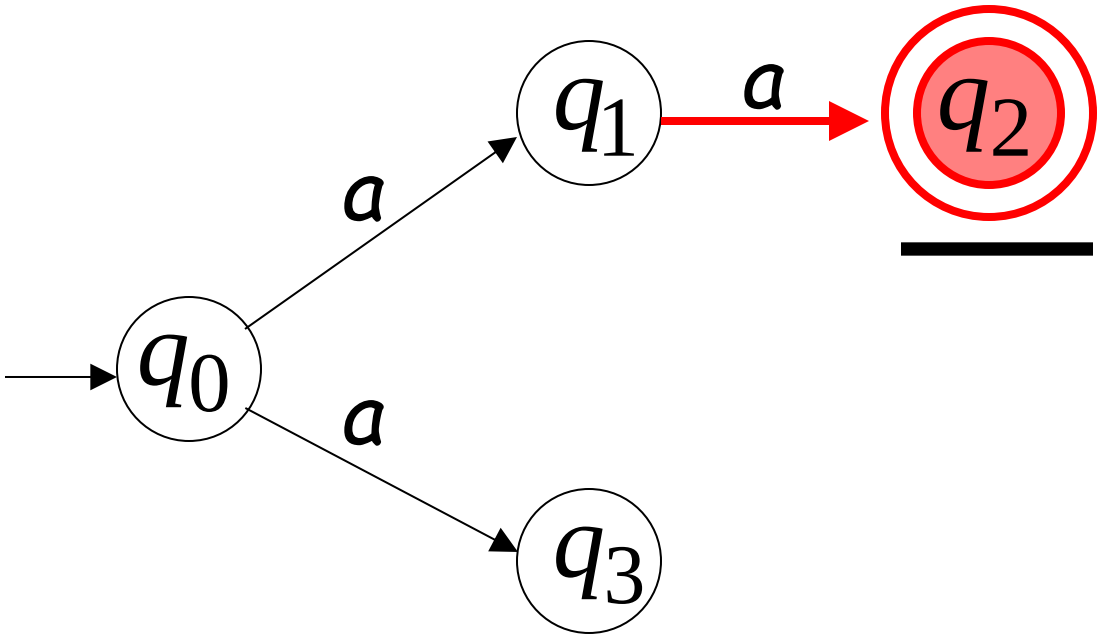
First Choice



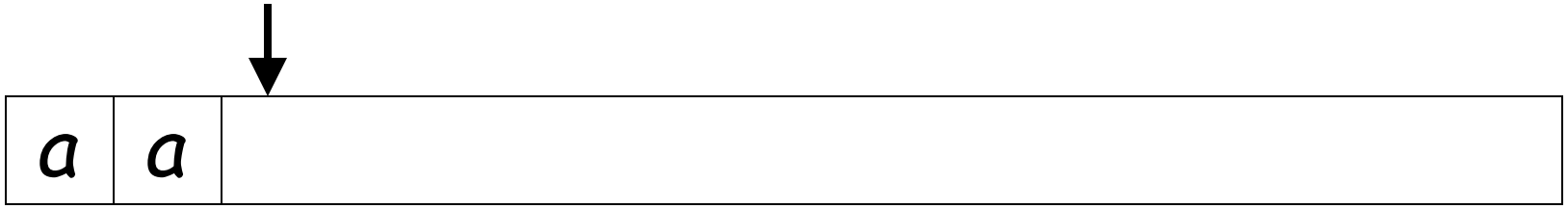


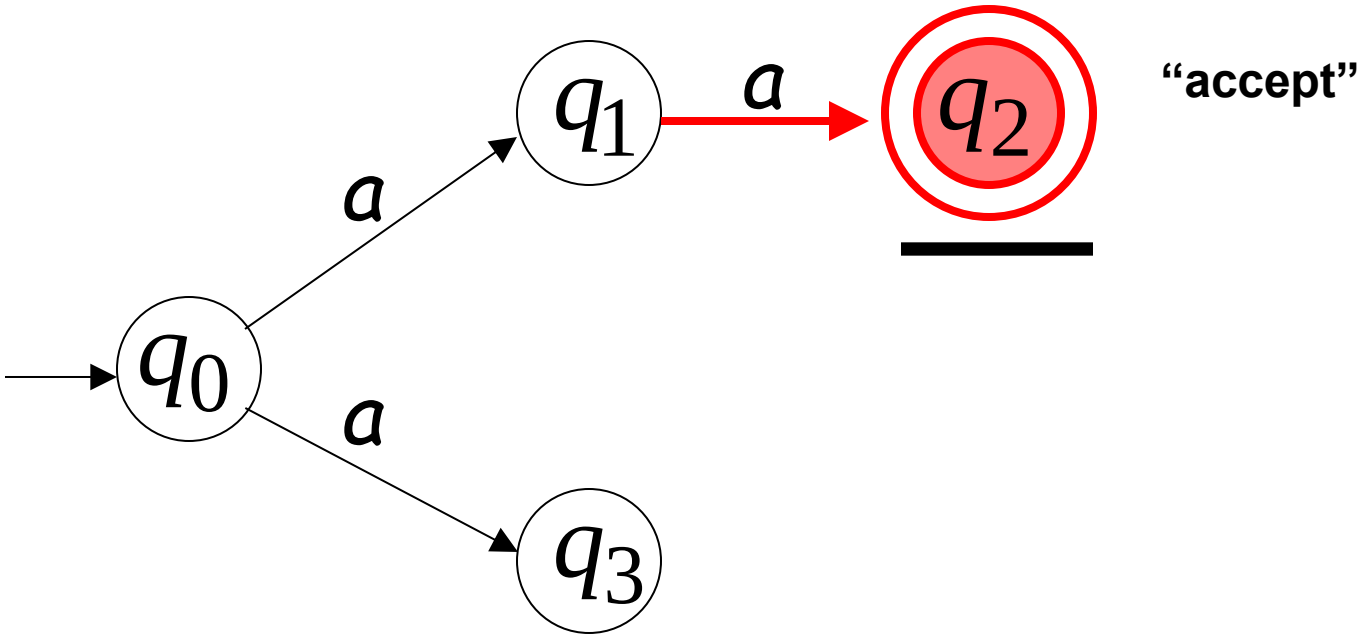
First Choice



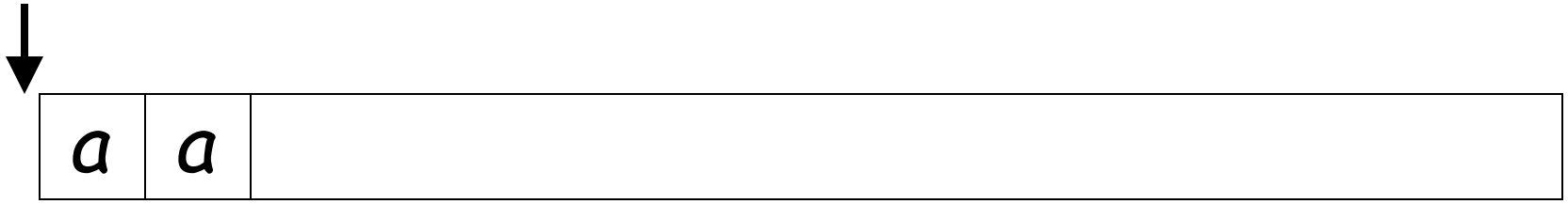


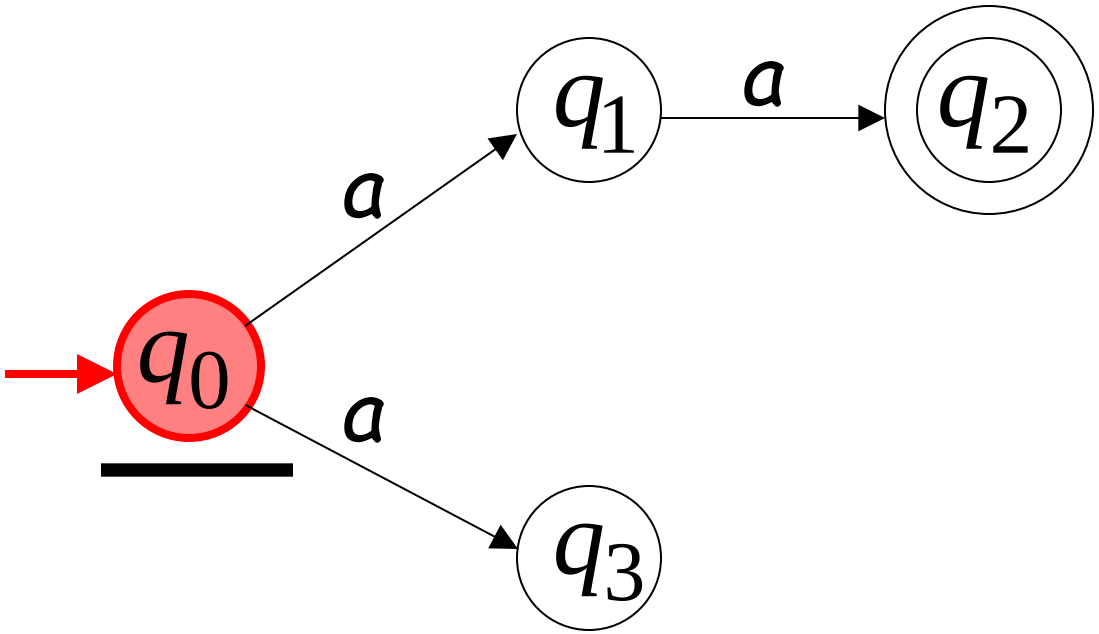
First Choice



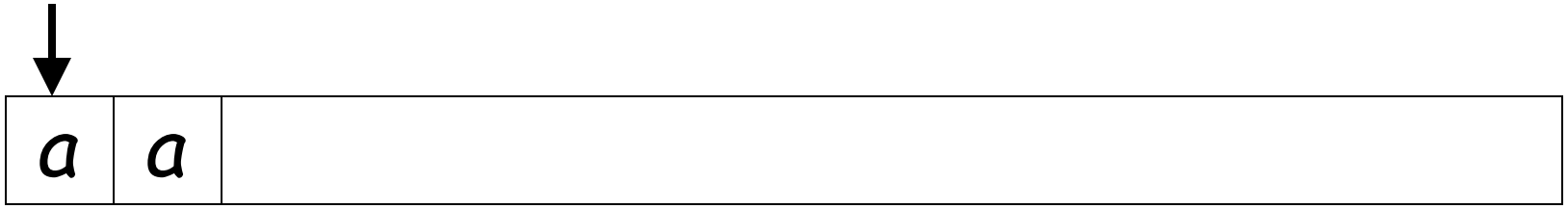


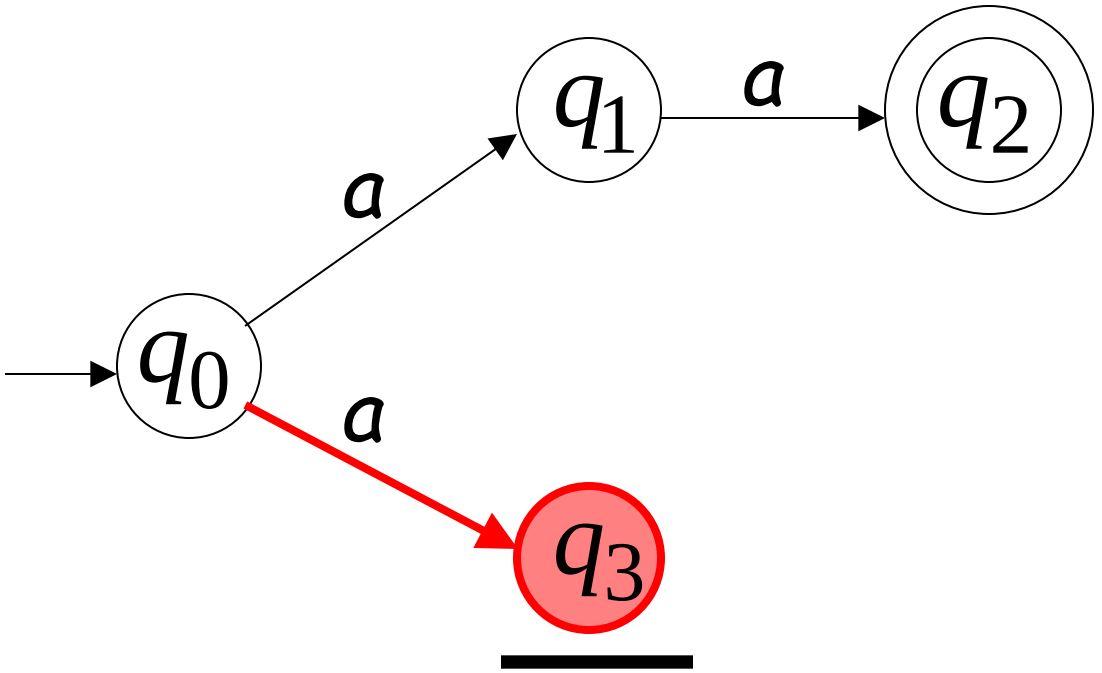
Second Choice



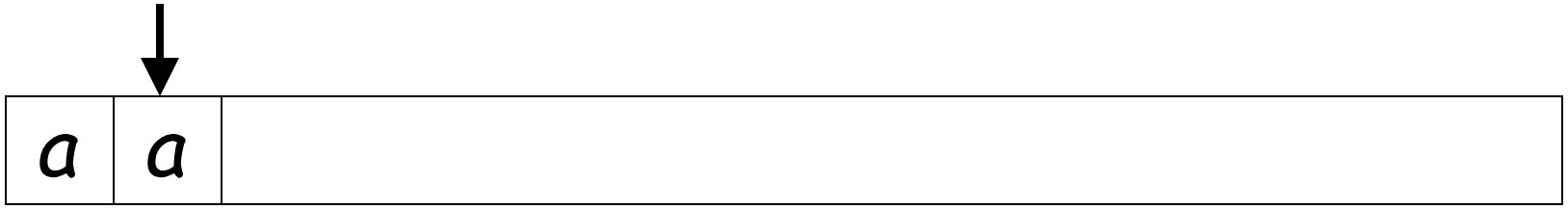


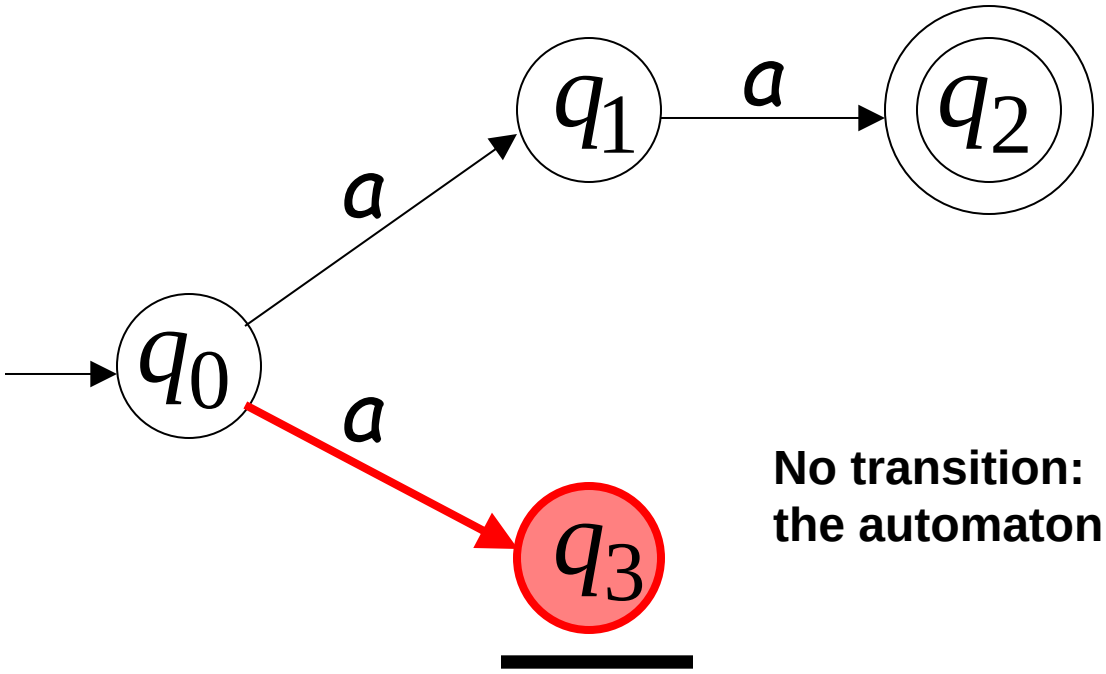
Second Choice





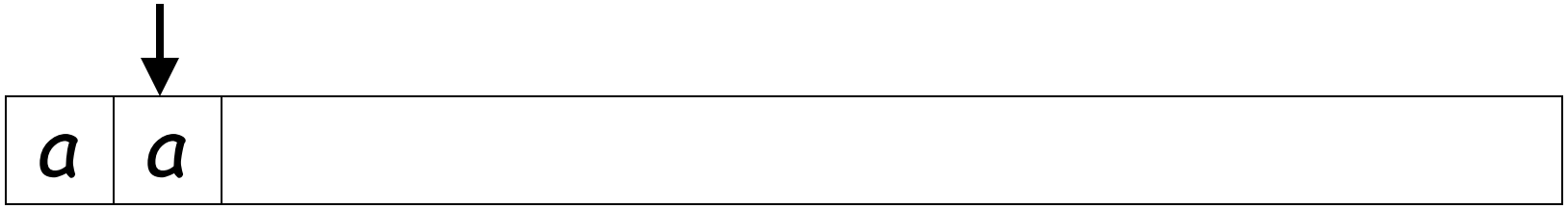
Second Choice

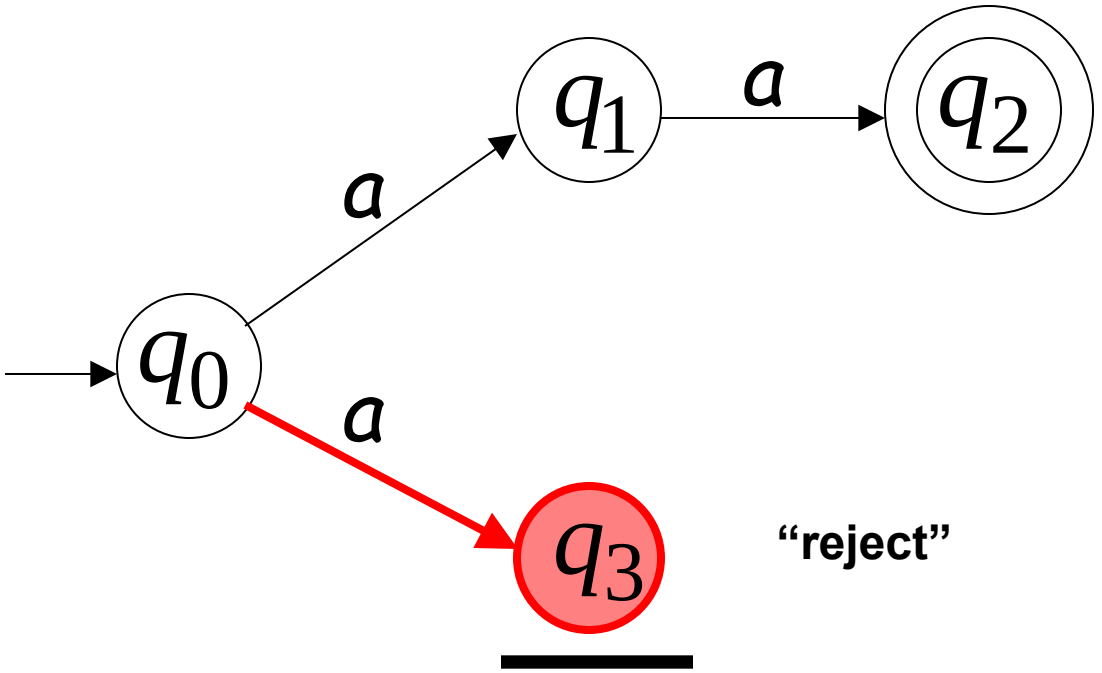




No transition:
the automaton hangs

Second Choice





"reject"

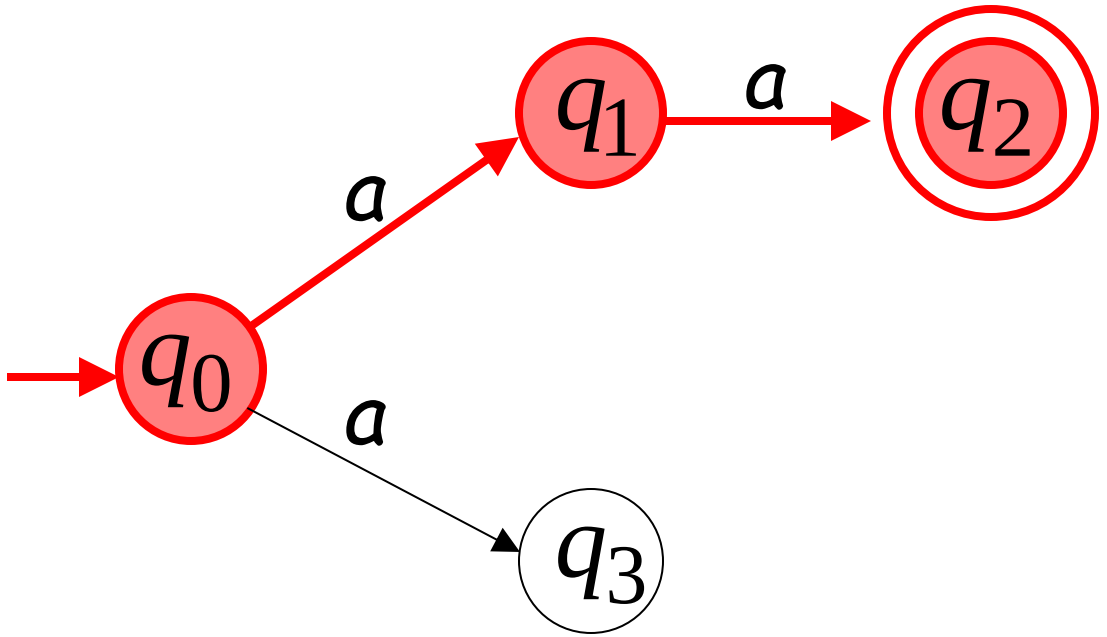
Observations

- An NFA accepts a string:

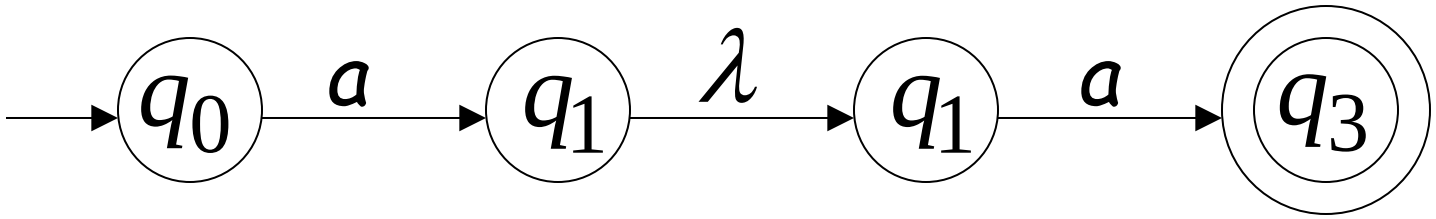
- If there is a computation of the NFA that accepts the string

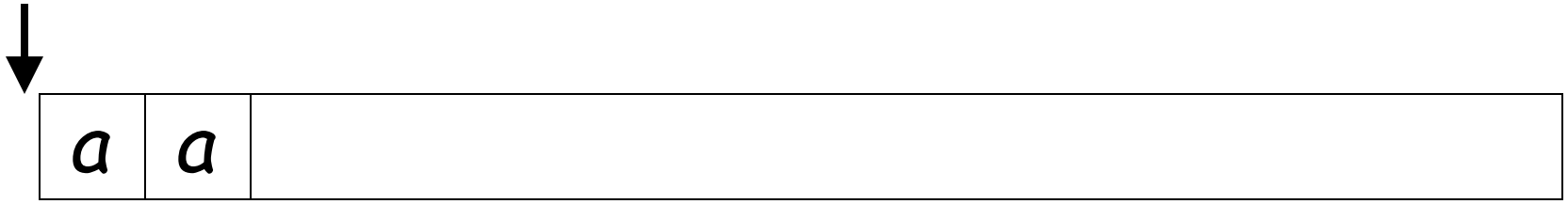
Example

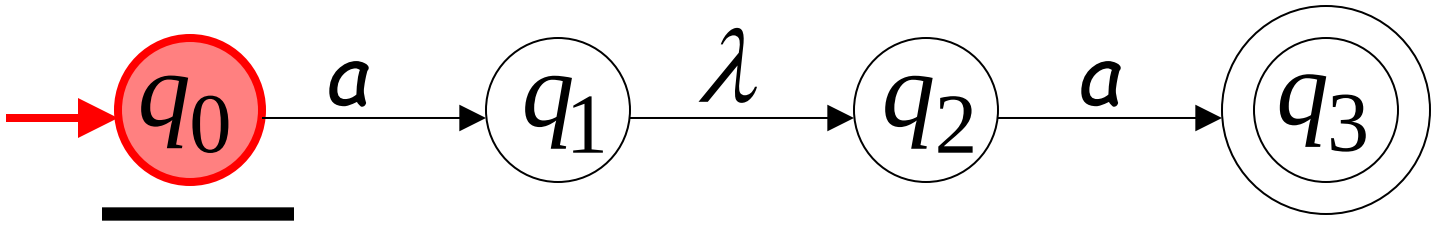
aa **is accepted by the NFA:**

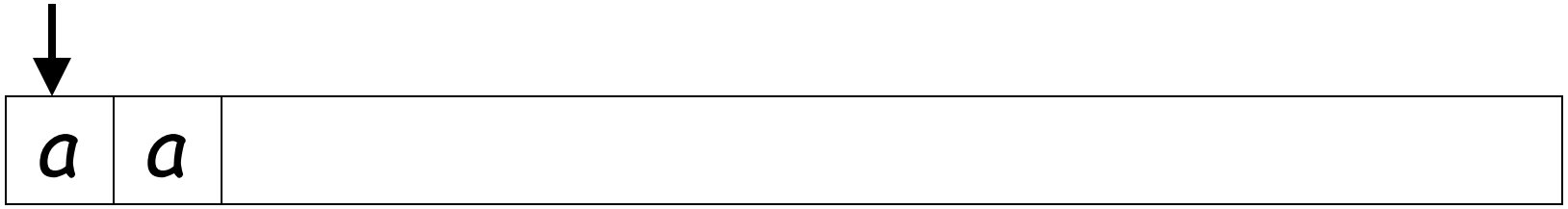


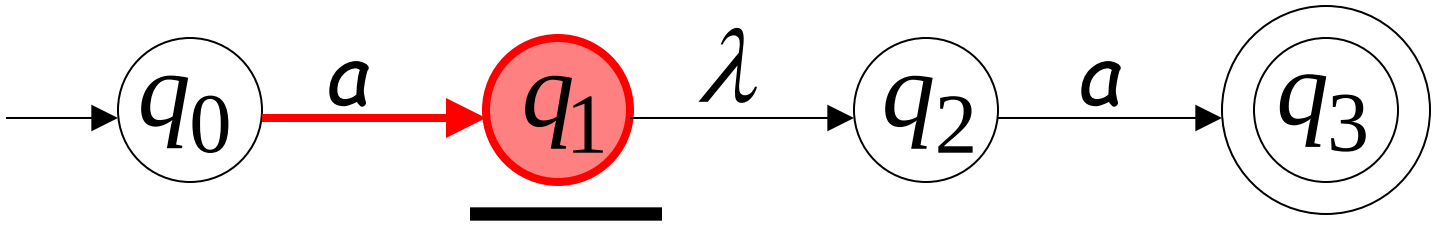
Lambda Transitions



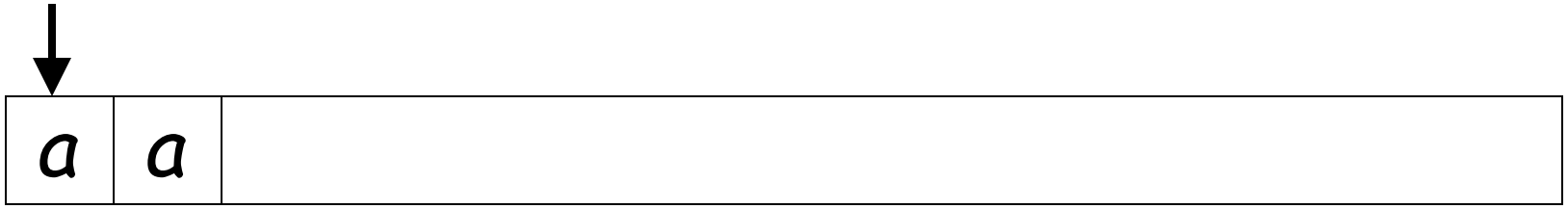

a
a

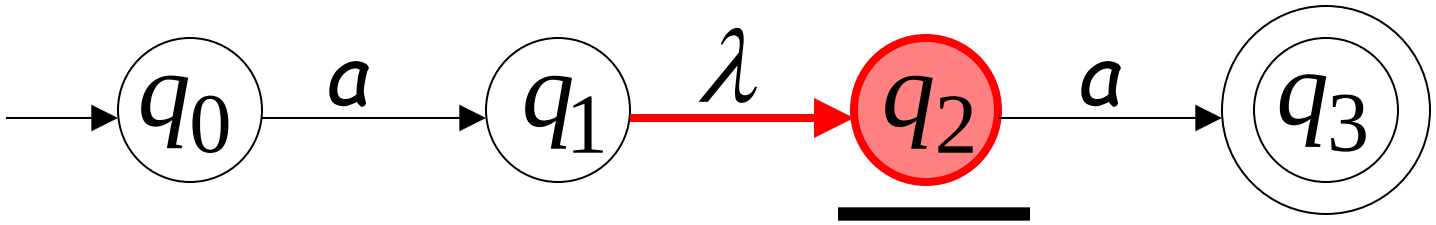

q_0
a
q_1
λ
q_2
a
q_3

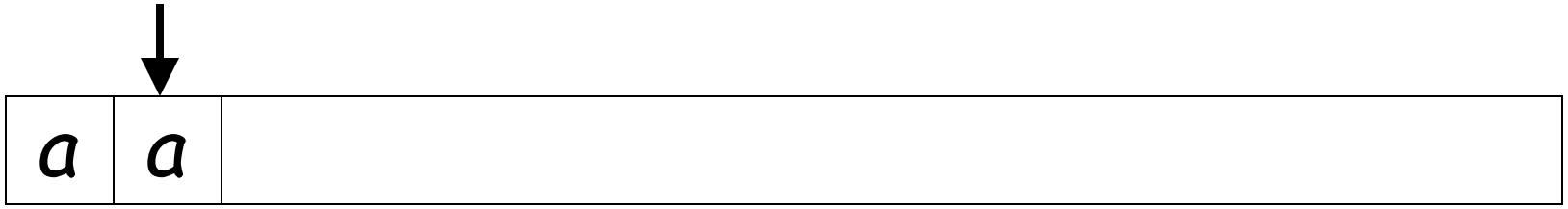

a
a

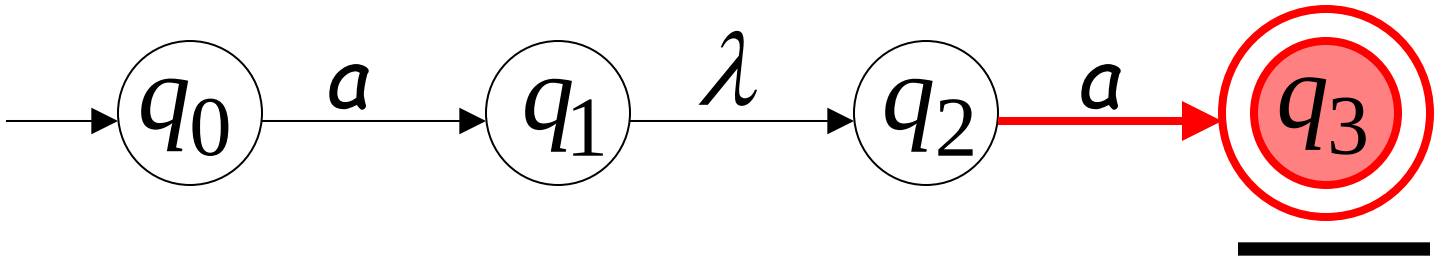

q_0
a
q_1
λ
q_2
a
q_3

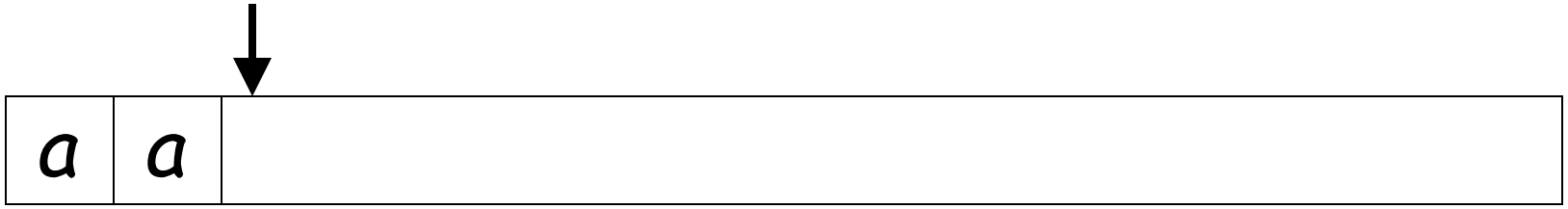
(read head doesn't move)

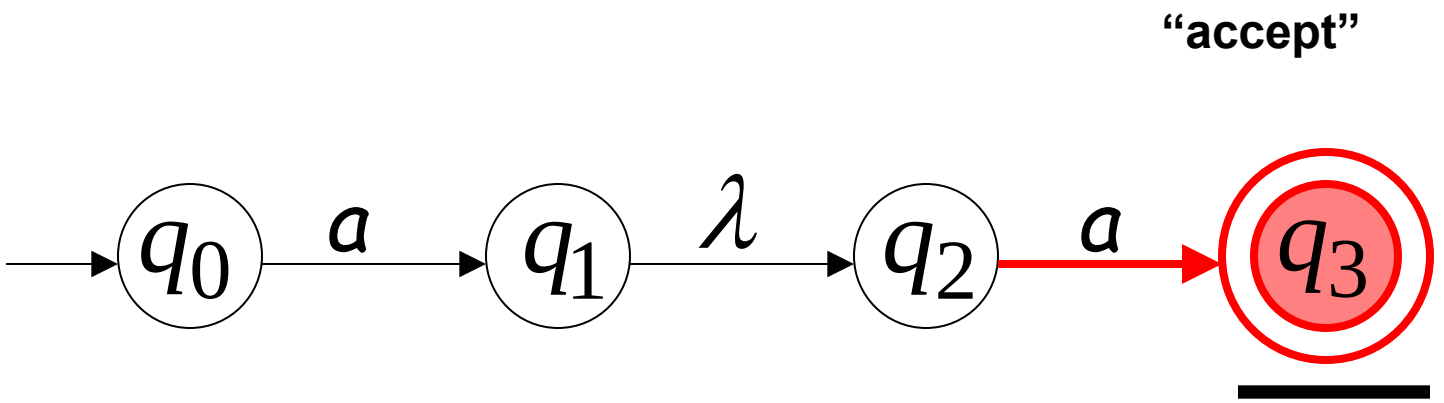





a
a

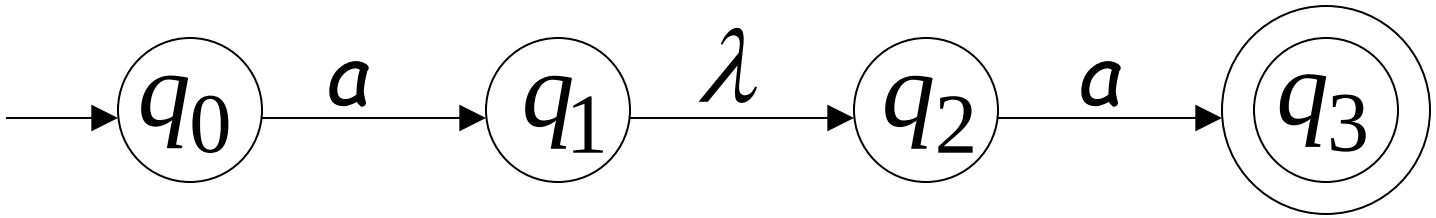

q_0
a
q_1
λ
q_2
a
q_3



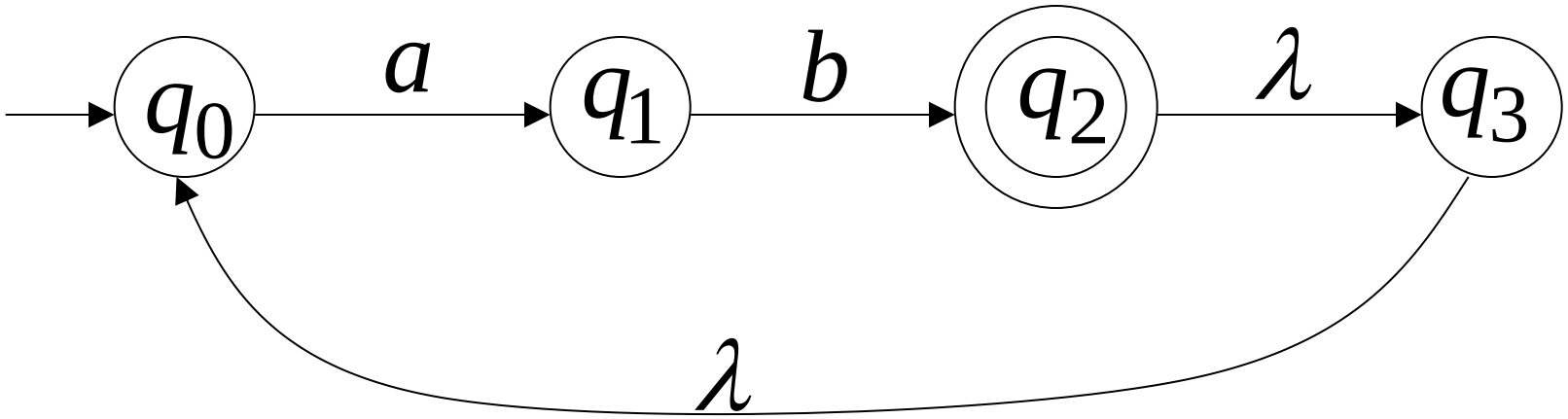


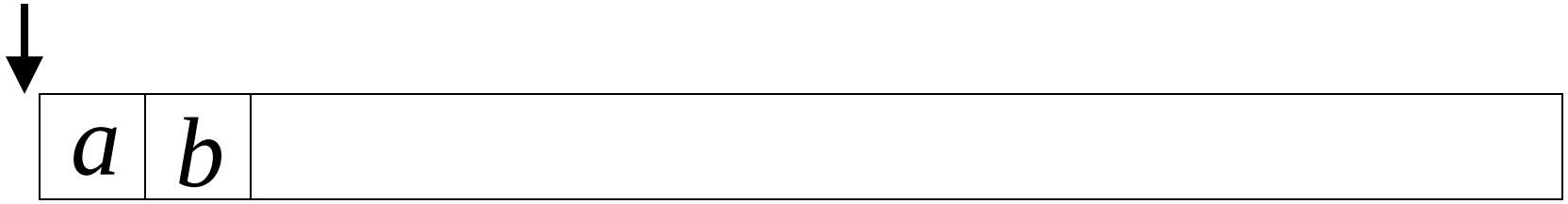
String aa is accepted

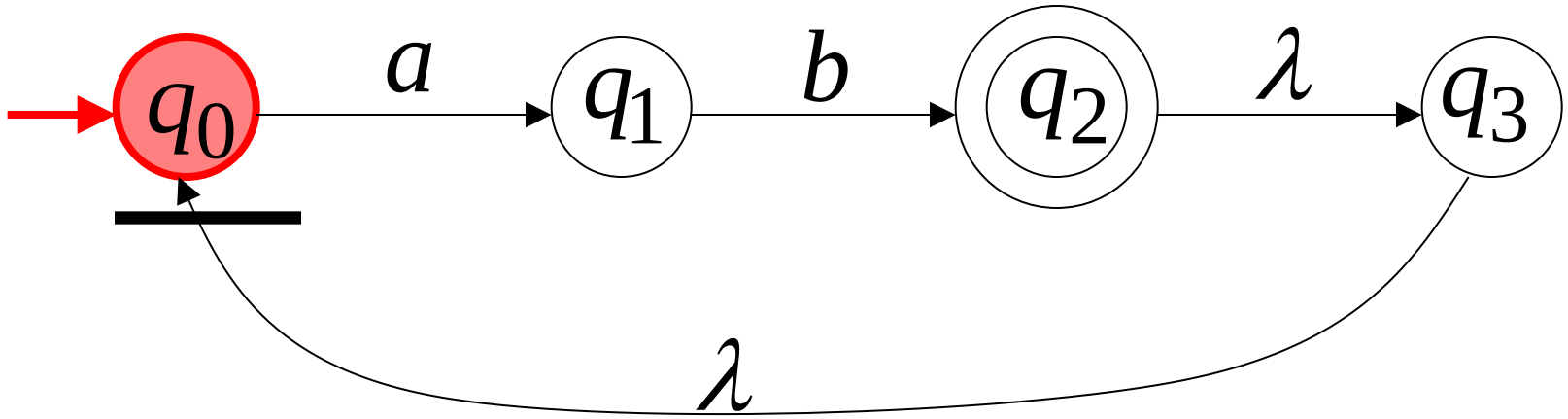
Language accepted: $L = \{aa\}$

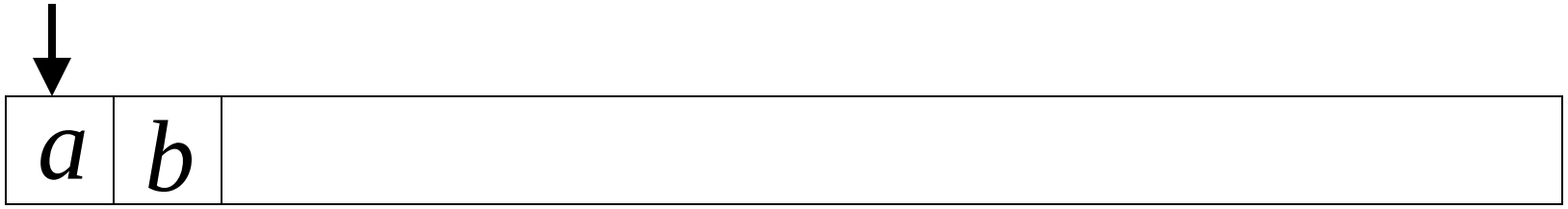


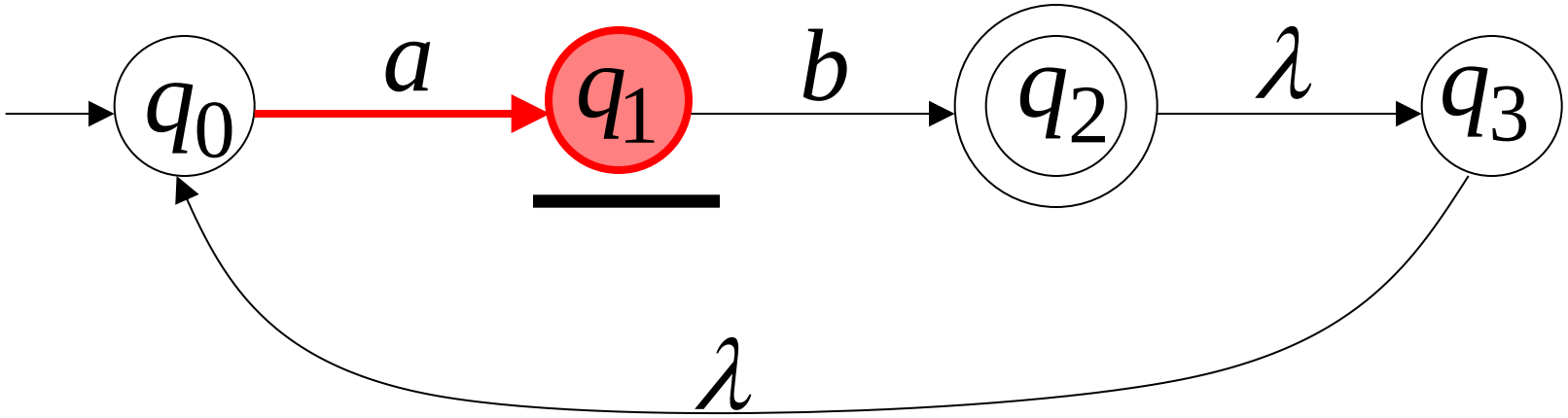
Another NFA Example

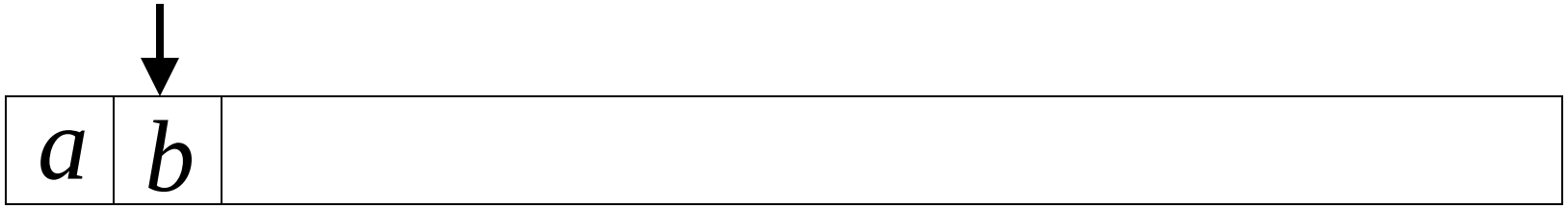


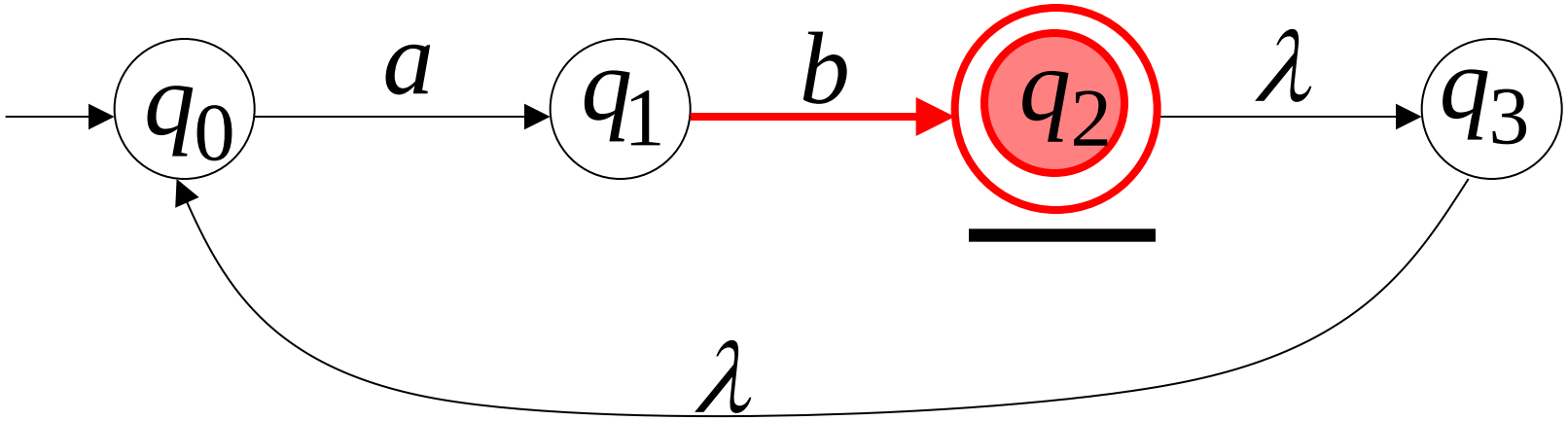

a
b

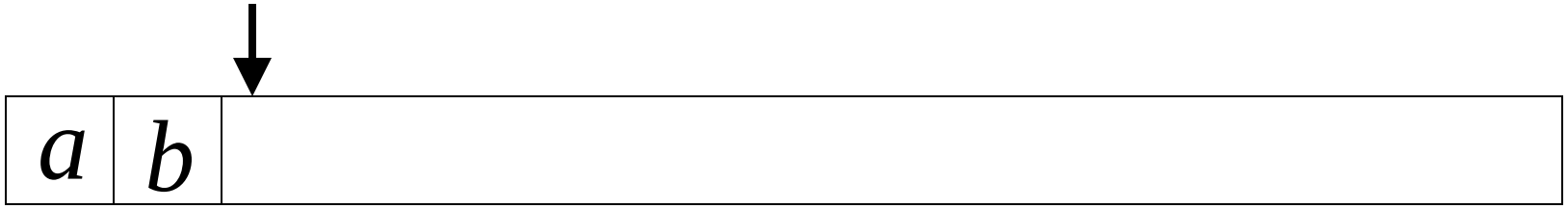

q_0
a
q_1
b
q_2
λ
q_3
λ

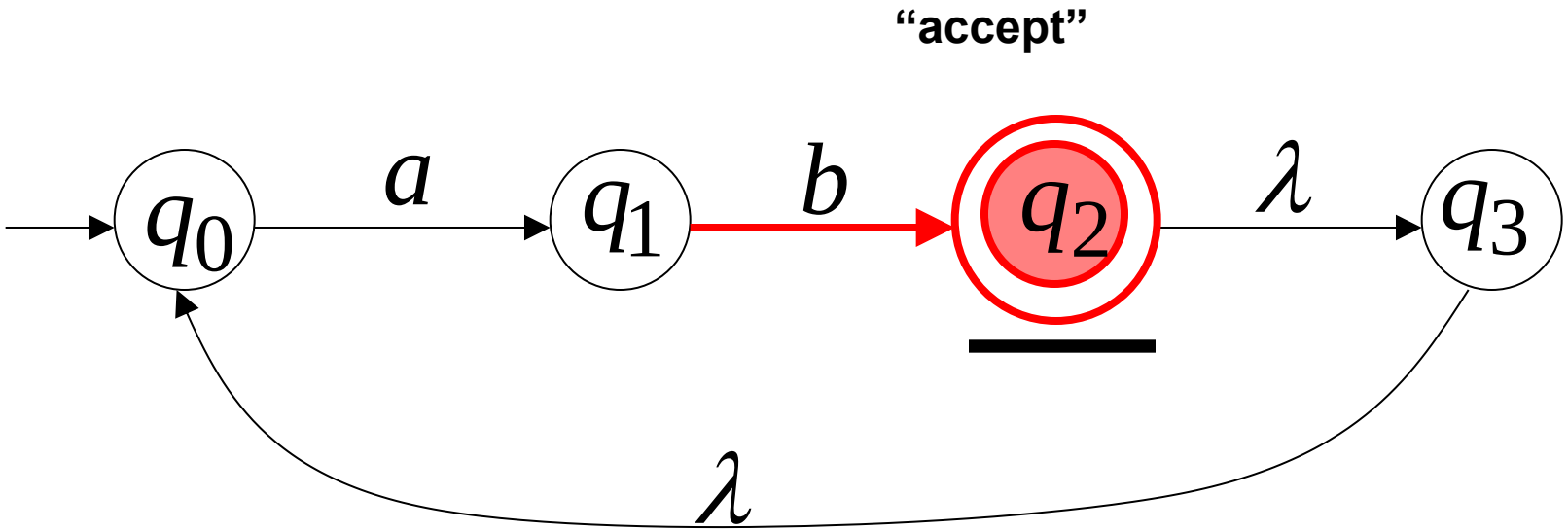

a
b


q_0
a
q_1
b
q_2
λ
q_3
λ

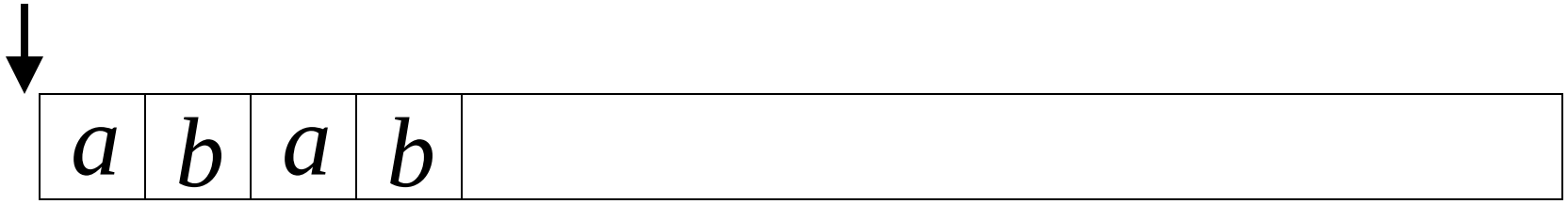

a
b

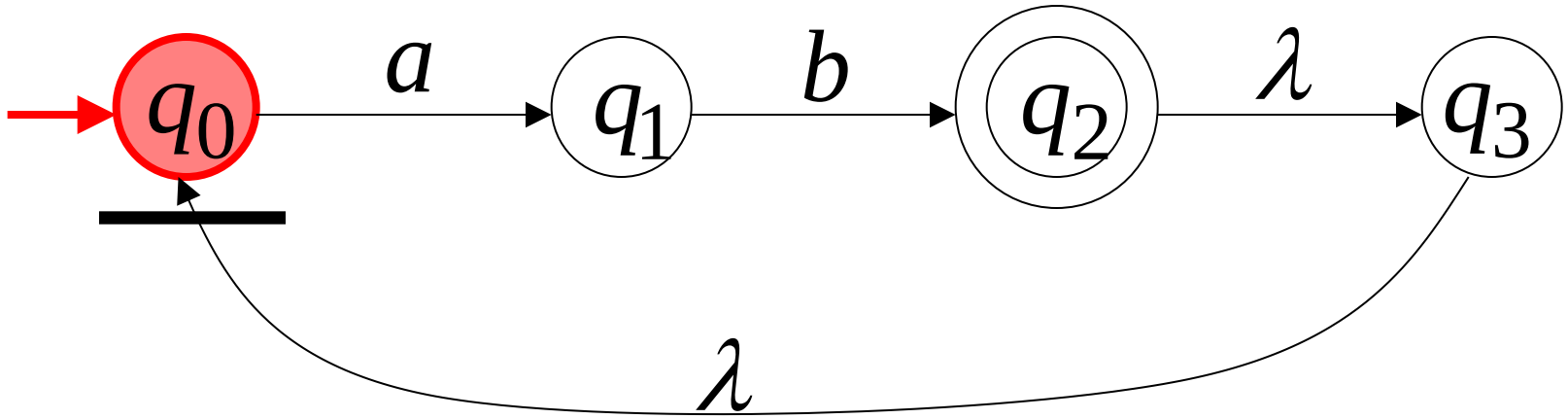

q_0
a
q_1
b
q_2
λ
q_3
λ

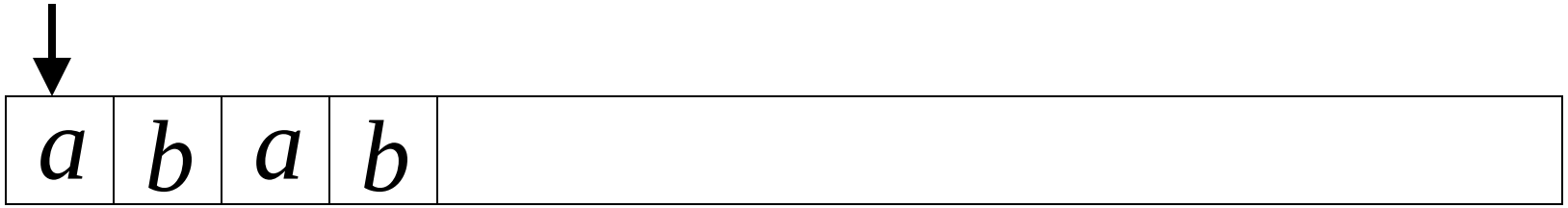

a
b

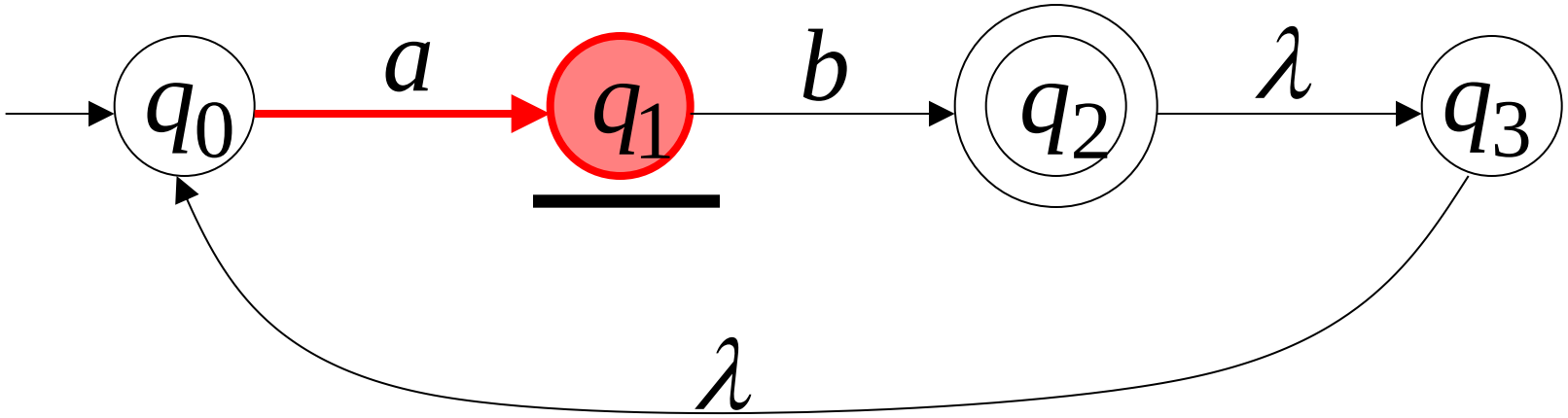

"accept"
q_0
a
q_1
b
q_2
λ
q_3
λ

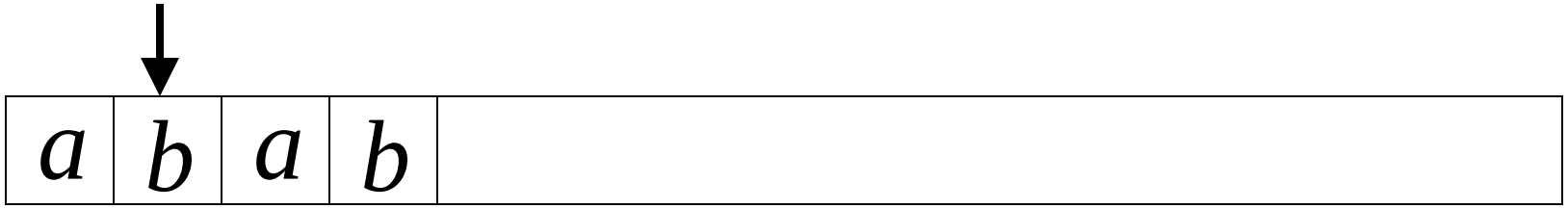
Another String

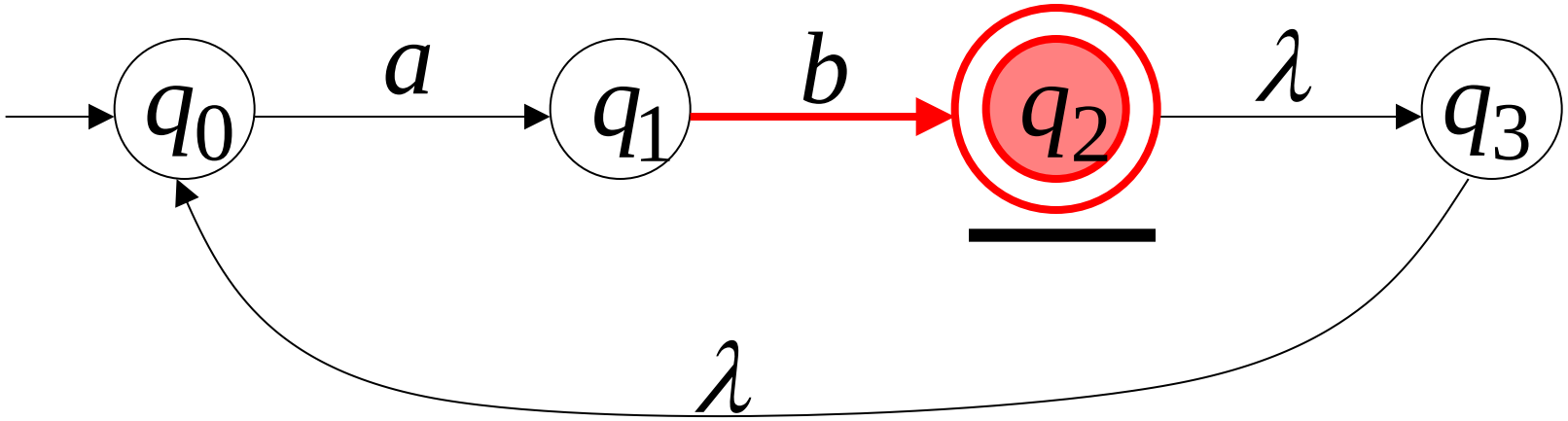


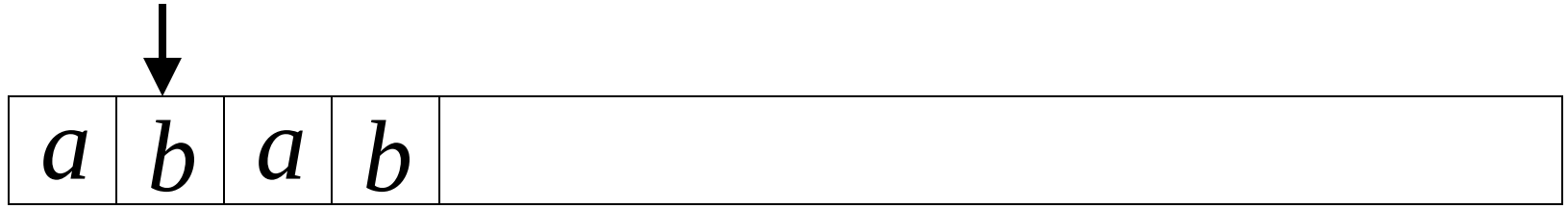


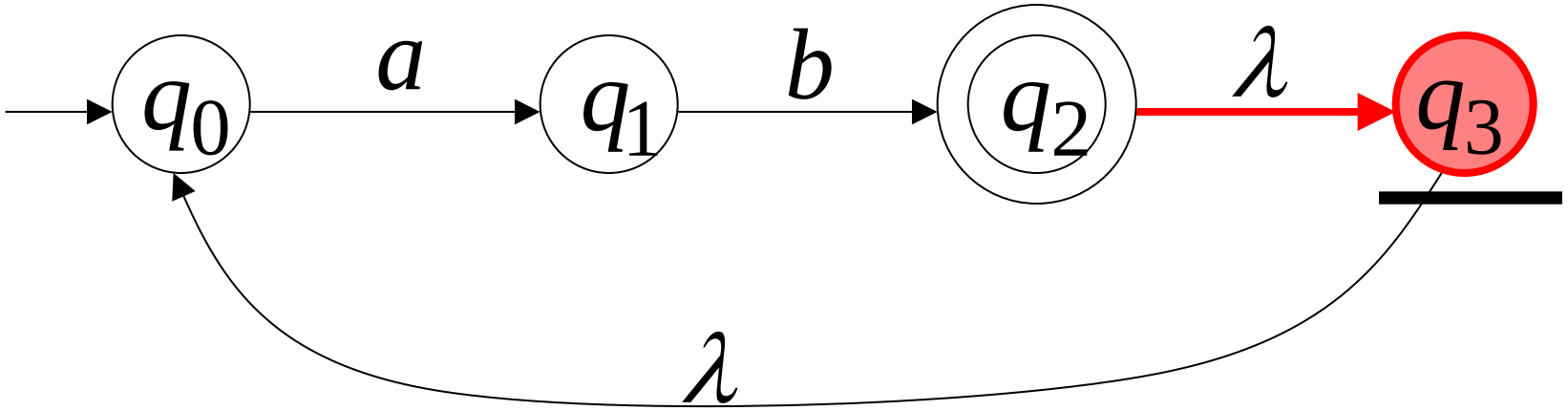

a
b
a
b

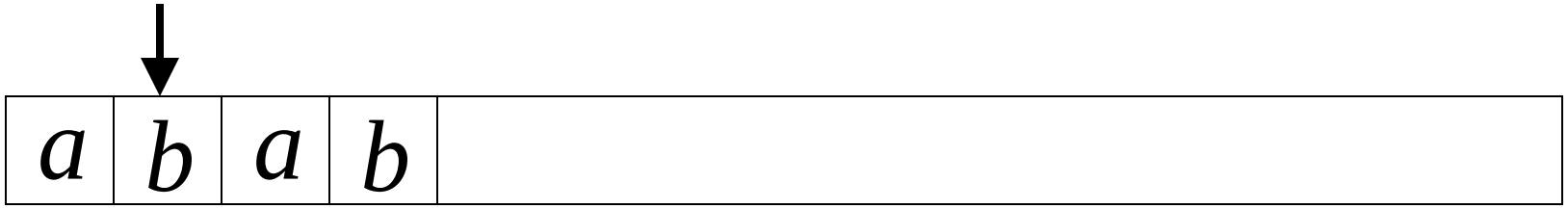

q_0
a
q_1
b
q_2
λ
q_3
λ

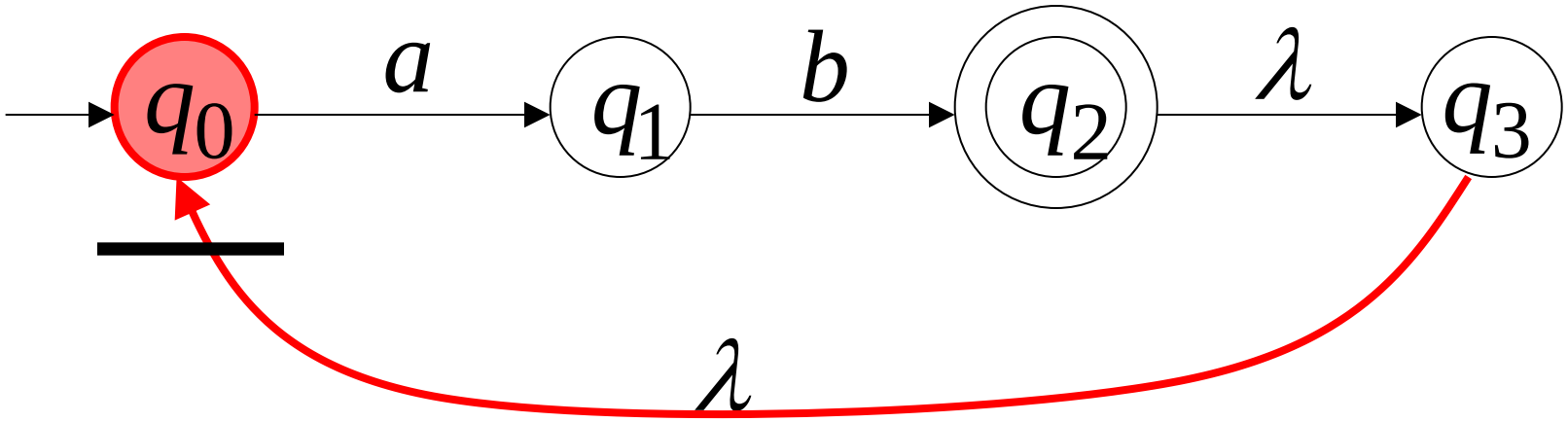

a
b
a
b

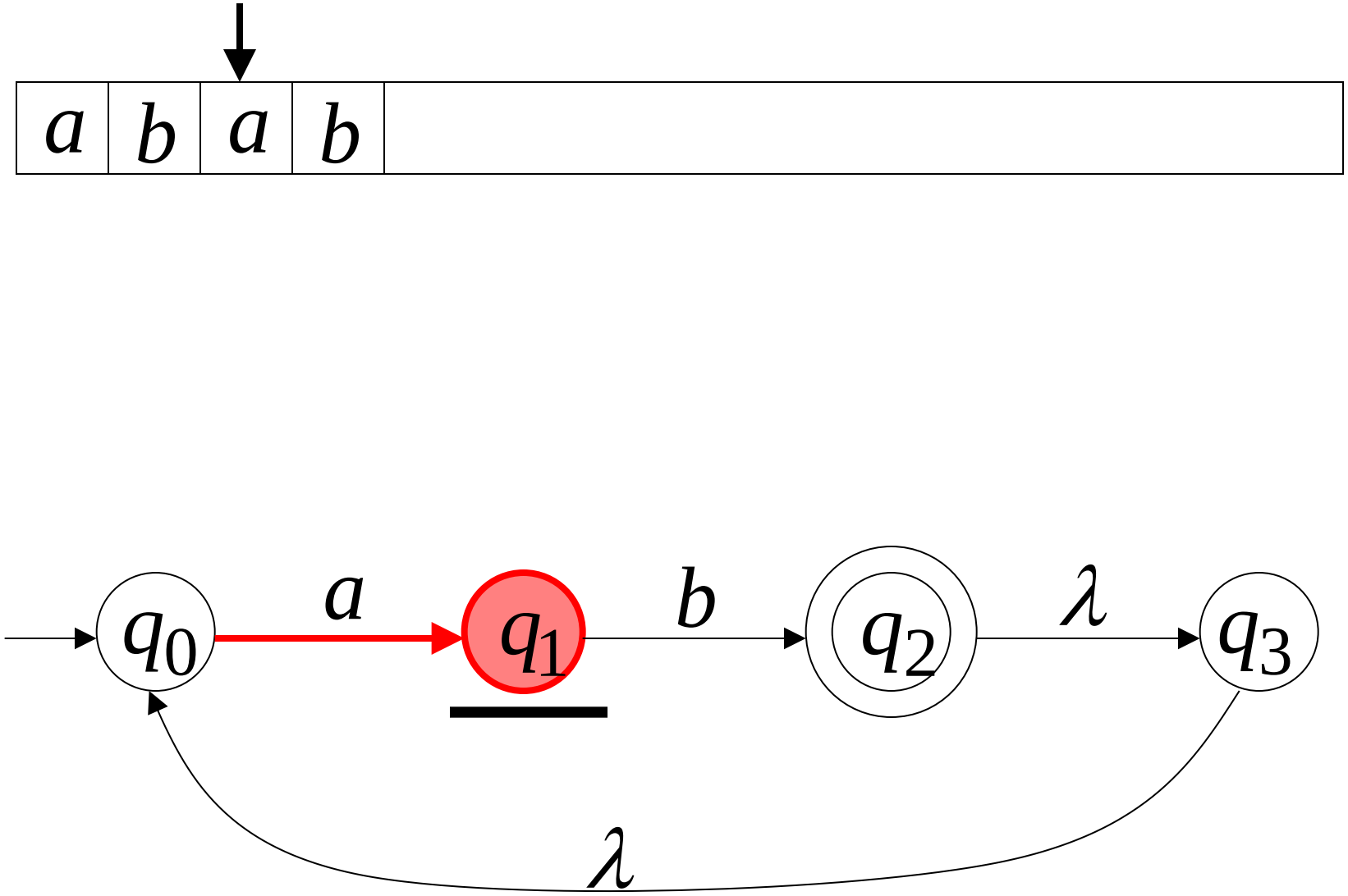

q_0
a
q_1
b
q_2
λ
q_3
λ

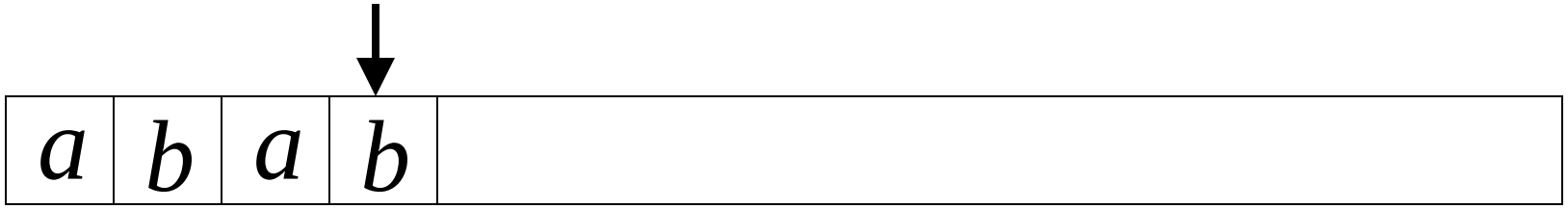

a
b
a
b

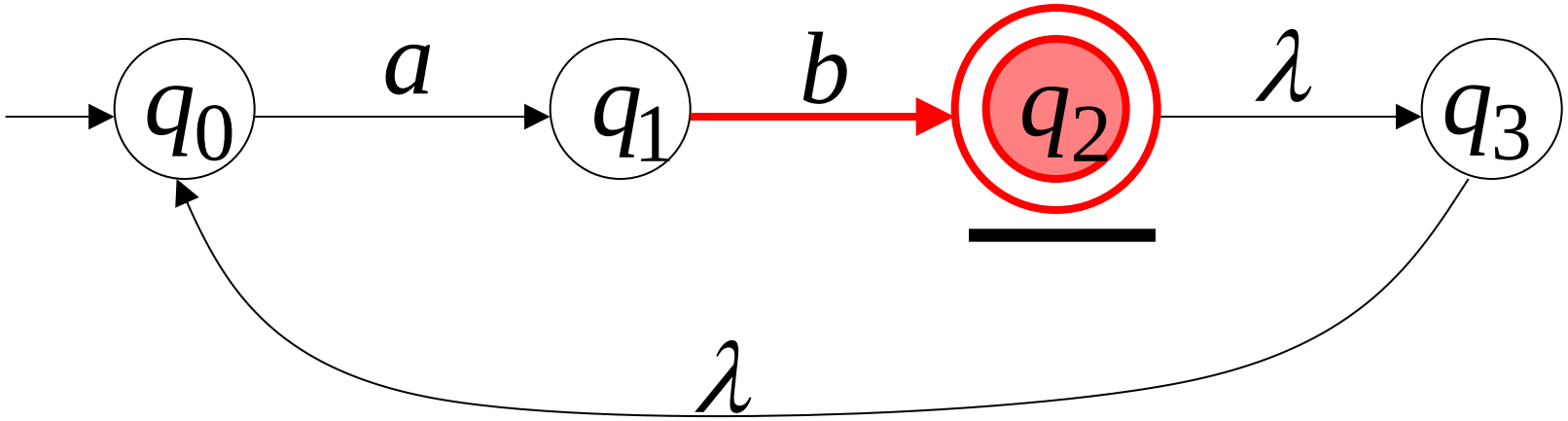

q_0
a
q_1
b
q_2
λ
q_3
λ

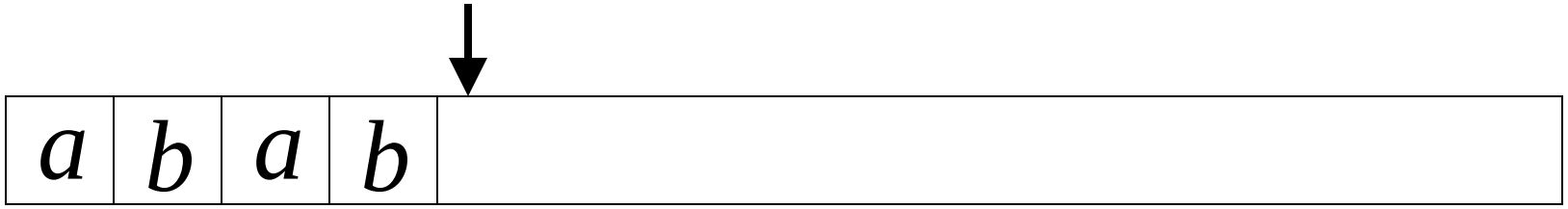

a
b
a
b

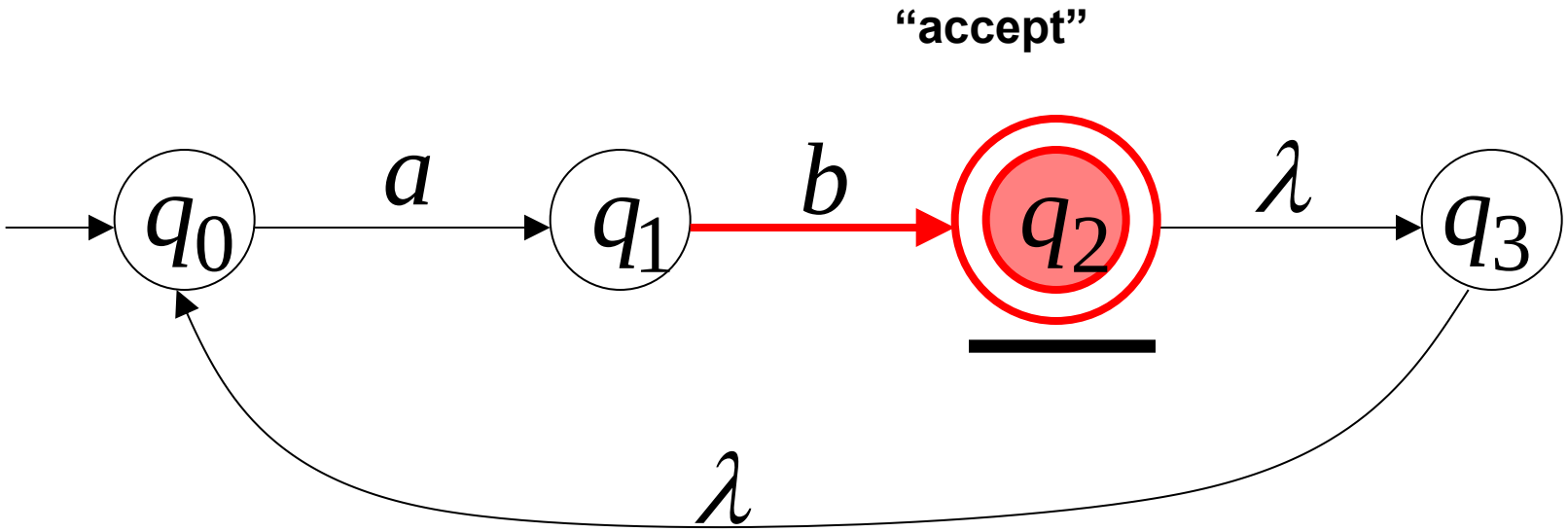

q_0
a
q_1
b
q_2
λ
q_3
λ


a
b
a
b
q0
a
q1
b
q2
λ
q3
λ


a
b
a
b


q_0
a
q_1
b
q_2
λ
q_3
λ

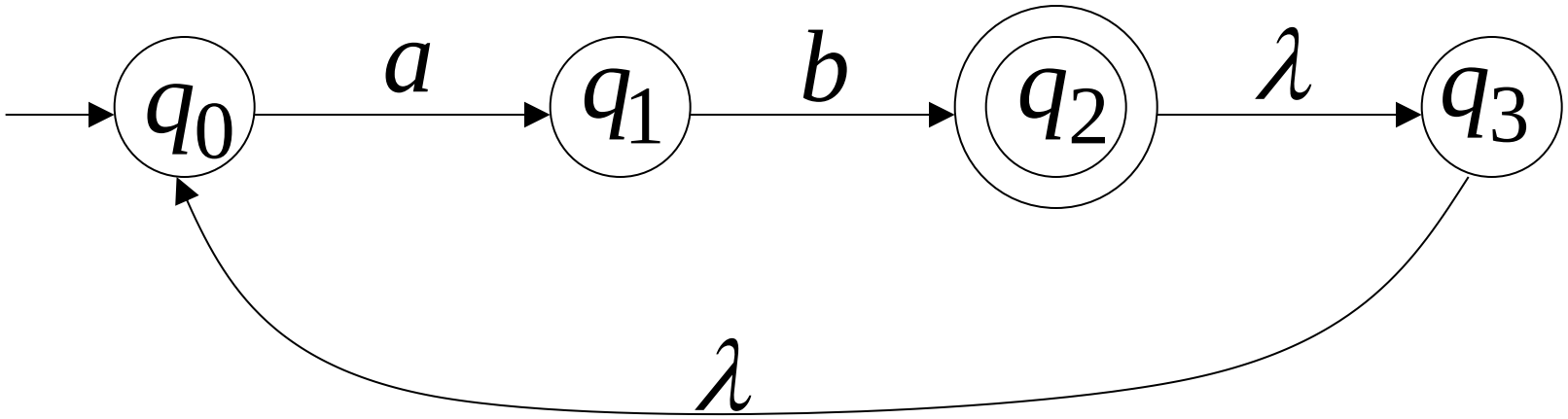

a
b
a
b


"accept"
q_0
q_1
q_2
q_3
a
b
λ
λ

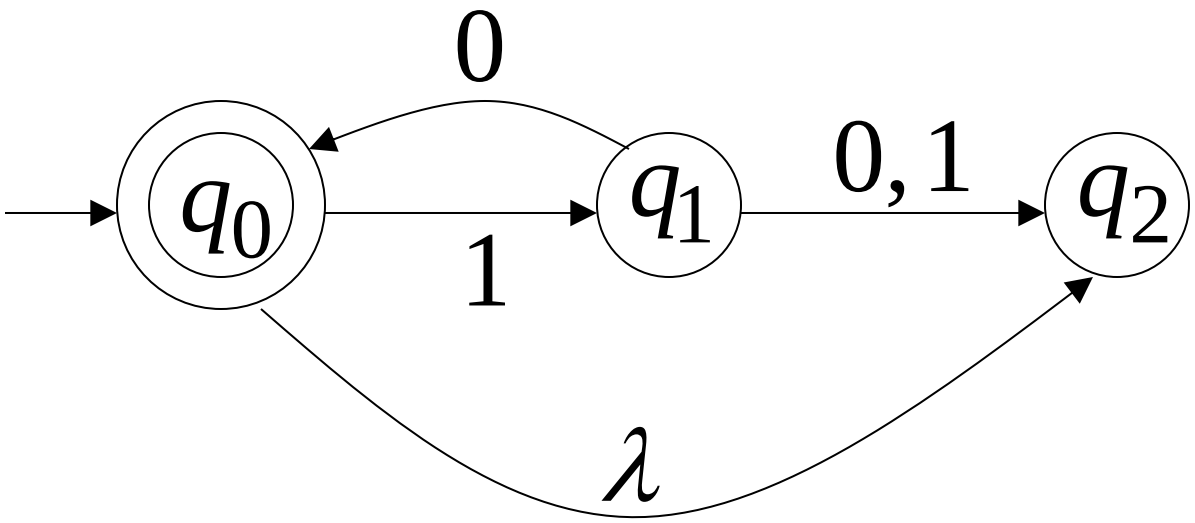
Language accepted

$$L = \{ab,\ abab,\ ababab,\ ...\}$$

$$= \{ab\}^{+}$$

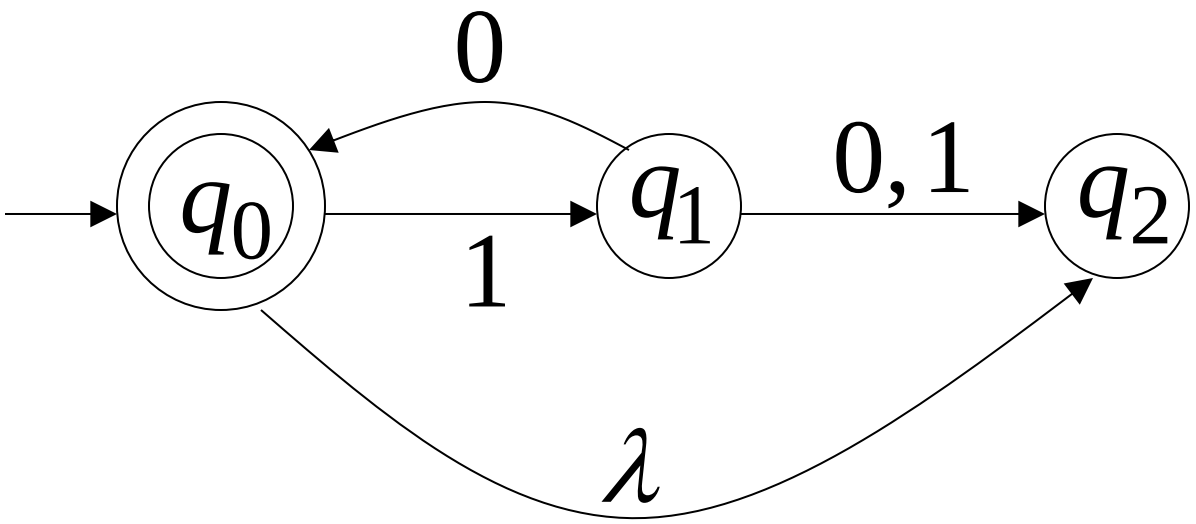


Another NFA Example

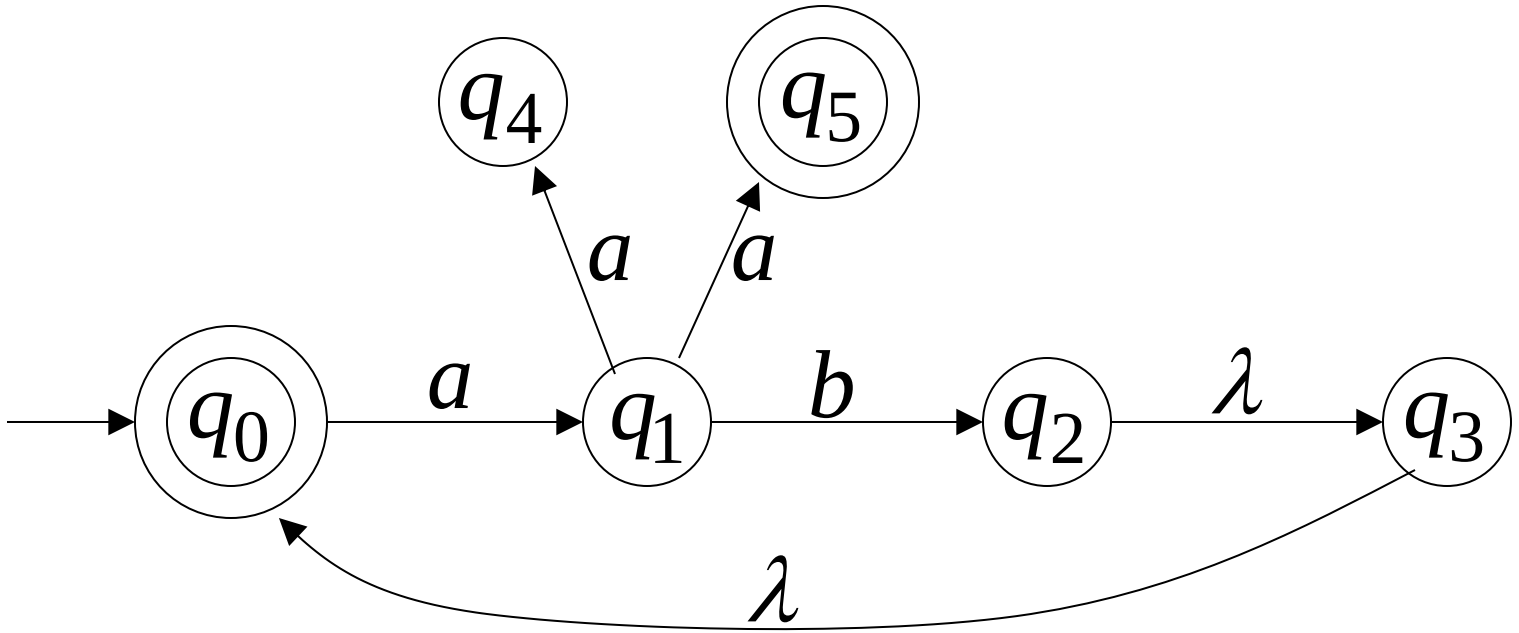


Language accepted

$$L = \{\lambda,\ 10,\ 1010,\ 101010,\ ...\}$$
$$= \{10\}*$$

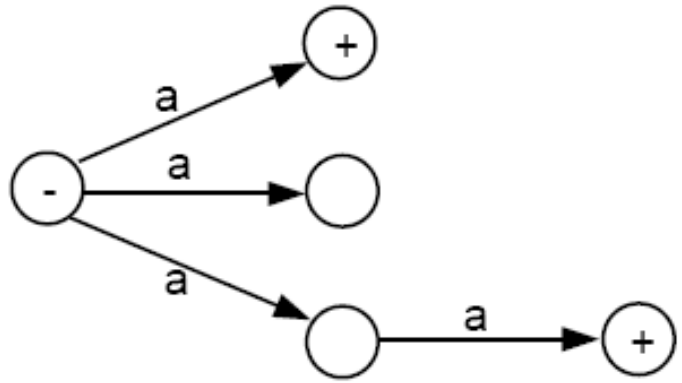


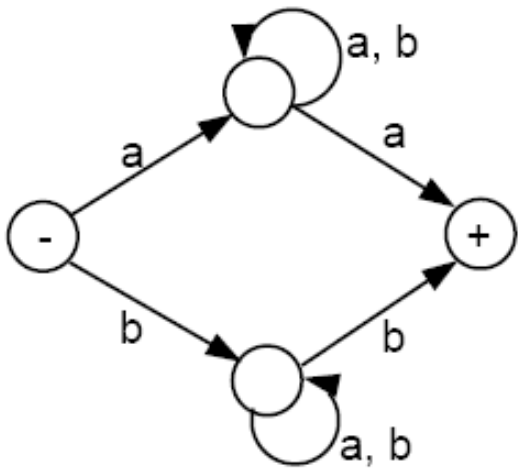
•

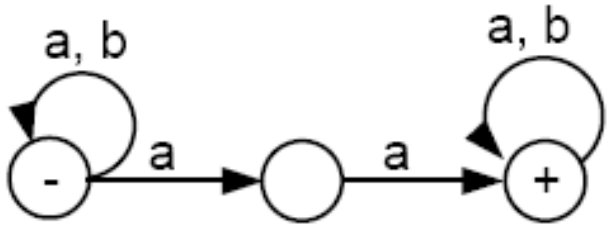


$$L(M) = \{aa\} \cup \{ab\}^* \cup \{ab\}^+\{aa\}$$

Examples of NFAs







Theorem 7

for every NFA, there is some FA that accepts exactly the same language.

- **Proof 1**

- By the proof of part 2 of Kleene's theorem, we can convert an NFA into a regular expression, since an NFA is a TG.

- By the proof of part 3 of Kleene's theorem, we can construct an FA that accepts the same language as the regular expression. Hence, for every NFA, there is a corresponding FA.

Distinguished Features

FA/DFA

1. One start state ONLY & Zero or more Final States

NFA

1. One start state ONLY & Zero or More Final States
2. Null transitions
3. Non-determinism

TG

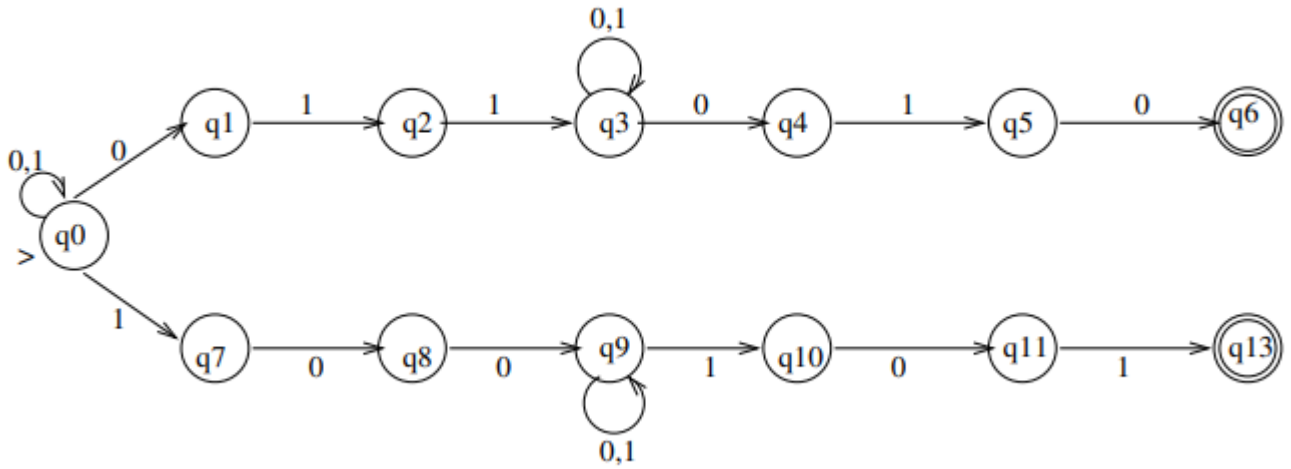
1. One or More Start States
2. Zero or More Final States
3. Multiple letters on the edges
4. Null transitions
5. Non-determinism

NFA

- Design an NFA (non-deterministic finite automata) to accept the set of strings of 0’s and 1’s that either (a) end in 010 and have 011 somewhere preceding, or (b) end in 101 and have 100 somewhere preceding

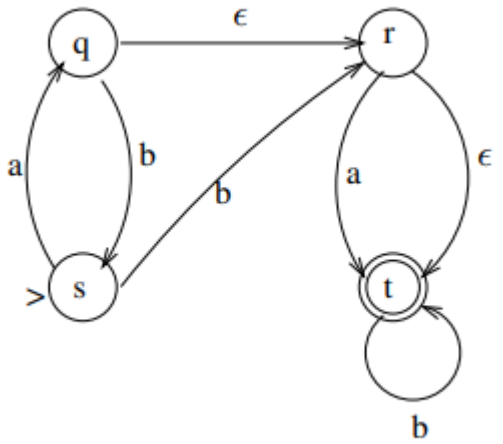
NFA

- Design an NFA (non-deterministic finite automata) to accept the set of strings of 0's and 1's that either (a) end in 010 and have 011 somewhere preceding, or (b) end in 101 and have 100 somewhere preceding



NFA to DFA Conversion

- Transform the following NFA (note that stands for null-string) into an equivalent DFA (deterministic finite automata)



NFA to DFA Conversion

- Transform the following NFA (note that stands for null-string) into an equivalent DFA (deterministic finite automata)